

Practica 2: Optimizacion conjunta calibracion-discriminacion, incertidumbre y servicio del modelo

Contexto

En clase hemos seguido construyendo el pipeline end-to-end de Machine Learning para deteccion de impago. Sobre la base de Practica 1 (preprocesamiento + filtrado + modelos), hemos visto en los siguientes notebooks de referencia:

1. **Optimizacion bayesiana de hiperparametros** ([11_optuna_tpe.ipynb](#)): uso de **Optuna** con sampler **TPE** y **MedianPruner** para tunear LightGBM, XGBoost y CatBoost minimizando **Brier score** sobre un split interno train/val. Incluye early stopping con metrica custom y comparacion contra los baselines del notebook 07.
2. **Calibracion de probabilidades** ([10_calibracion.ipynb](#)): diagnostico con **reliability diagram**, **Brier score** y **Expected Calibration Error (ECE)**, y aplicacion de **Platt scaling** (sigmoid) e **isotonic regression** mediante **sklearn.calibration.CalibratedClassifierCV**. Reflexion sobre **cuando calibrar** y **cuando no merece la pena** (modelos ya calibrados, datasets pequenos, etc.).
3. **Conformal Prediction** ([09_conformal_prediction.ipynb](#)): construccion de **intervalos de prediccion** con cobertura garantizada (Inductive Conformal Prediction, split-conformal). Permite cuantificar la **incertidumbre** de cada prediccion y derivar las decisiones inciertas a un humano (o a un agente especializado).

Importante - punto de partida comun: para que todos partamos exactamente del mismo dataset y los resultados sean comparables, esta practica **debe partir de los pickles ya preprocesados y filtrados** que utiliza el notebook 11:

- [data/filtered/X_train_filtered.pkl](#)
- [data/filtered/y_train_filtered.pkl](#)
- [data/filtered/X_test_filtered.pkl](#)
- [data/filtered/y_test_filtered.pkl](#)

Estos artefactos se generan al final del notebook

[06_clean_preprocessing_and_filtering_model.ipynb](#) (que ya aplica el preprocesado sobre [data/variables_withExperts.xlsx](#) y el filtrado de features). **No se debe re-preprocesar ni re-filtrar** en esta practica: se carga el pickle y se trabaja sobre el. Asi todos los alumnos optimizan, calibran y miden incertidumbre sobre la misma matriz de features y se pueden comparar resultados de forma justa.

Objetivo de la practica

Construir, sobre el pipeline ya conocido, un **flujo completo de modelado y servicio** que incluya:

1. **Optimizacion automatica** de hiperparametros con Optuna usando una metrica que **mejore la calibracion sin degradar la discriminacion**.

2. **Calibracion** de las probabilidades del mejor modelo, justificando si es necesario o no.
3. **Cuantificacion de incertidumbre** mediante conformal prediction y politica de **derivacion a un agente** cuando la prediccion no es fiable.
4. **API local** con dos servicios (subida de modelo + prediccion) lista para servir el modelo.
5. **Aplicacion web local** que use el endpoint para estimar el riesgo de impago de un cliente nuevo, pidiendo solo las **5 variables mas relevantes** segun un Random Forest.

Entrega

La practica se entregara como **tres repositorios Git independientes** (GitHub, GitLab o similar). Se debe entregar **una URL por repositorio**. **No es necesario desplegar nada en la nube**: basta con que todo se pueda ejecutar en local siguiendo el **README** de cada repo.

Repo	Contenido	Stack principal
Repo 1 - Modelado	Notebook con Optuna, calibracion, medida de incertidumbre y persistencia del modelo final (.pkl)	Python + scikit-learn + Optuna + (Venn-Abers, etc.)
Repo 2 - API	API REST que carga el .pkl y sirve predicciones con politica de derivacion	FastAPI + uvicorn
Repo 3 - Web	Mini web que llama a la API, pide solo las 5 features mas relevantes y muestra la prediccion	HTML + JS plano (o equivalente sencillo)

Requisitos comunes a los tres repos:

- Accesibles por el profesor (publicos o con permisos de lectura).
- **README** con instrucciones claras de instalacion y ejecucion en local.
- Se entrega cada repo con su artefacto final: el **.pkl** del modelo (Repo 1), la API arrancable (Repo 2) y la web abrible en navegador (Repo 3).
- En el Repo 1, los notebooks deben subirse **ya ejecutados** (con las celdas de salida visibles).

Contenido de cada repositorio

Repo 1 - Modelado: notebook **practica2_notebook.ipynb**

Notebook que ejecute, de principio a fin, los pasos de modelado, calibracion e incertidumbre, y guarde al final el artefacto del modelo que consumira la API. Estructura sugerida:

1.1 Optimizacion con Optuna: calibracion y discriminacion (variacion al notebook 11)

El notebook 11 optimizaba **Brier score** (que penaliza calibracion + resolucion juntas) y los notebooks anteriores (07) miraban solo AUC. Aqui queremos optimizar una metrica que **mejore la calibracion sin degradar la discriminacion**.

- **Cargar directamente** los pickles **data/filtered/{X_train_filtered, y_train_filtered, X_test_filtered, y_test_filtered}.pkl** (mismos artefactos que el notebook 11). No re-preprocesar ni re-filtrar.

- Optimizar al menos **dos modelos** (de LightGBM, XGBoost, CatBoost o equivalente) con **Optuna + TPE** y aplicar las siguientes **variaciones** respecto al notebook 11:
 - **Metrica objetivo:** usar una metrica que capture ambas dimensiones. Elegir **una** de las siguientes opciones (justificar la eleccion):
 1. **Log Loss** (`sklearn.metrics.log_loss`): proper scoring rule, descomponible en **resolution + reliability**. Penaliza tanto la mala discriminacion como la mala calibracion. Es la opcion mas sencilla y la recomendada por defecto.
 2. **Optimizacion multi-objetivo** con Optuna (`directions=['minimize', 'maximize']`): minimizar **Brier** o **ECE** y maximizar **AUC** simultaneamente. Optuna devuelve el **frente de Pareto**. Hay que documentar como se elige el modelo final del frente (criterio explicito, ej. "menor Brier entre los modelos con $AUC \geq 0.97 * AUC_{max}$ ").
 3. **Metrica combinada** custom, ej. $AUC - \lambda * ECE$ o $MCC - \lambda * Brier$, con λ razonado.
 - **Sampler/pruner:** cambiar al menos uno de los siguientes parametros respecto al notebook 11. Por ejemplo: **HyperbandPruner** o **SuccessiveHalvingPruner** en lugar de **MedianPruner**, o cambiar `n_startup_trials / multivariate` del **TPESampler**. Justificar el cambio.
 - **Espacio de busqueda:** anadir o ampliar al menos un hiperparametro respecto al notebook 11 (ej. `min_child_weight` en XGBoost, `feature_fraction_bynode` en LightGBM, `border_count` en CatBoost).
 - **Comparacion balanceado vs no balanceado:** para cada modelo, lanzar el **study dos veces**:
 1. Con tratamiento de desbalanceo (`class_weight='balanced', scale_pos_weight=neg/pos` o `auto_class_weights='Balanced'`).
 2. Sin tratamiento de desbalanceo.

Comparar el **best_value** (la metrica elegida) y, sobre todo, las metricas finales en test. Comentar que opcion gana en calibracion (Brier/ECE) y que ocurre con discriminacion (AUC/MCC). **Importante:** el balanceo de clases sesga las probabilidades; comentar el efecto sobre la calibracion.
- Para cada modelo final, calcular en test y mostrar en una tabla: **Accuracy, Precision, Recall, F1, MCC, ROC-AUC, PR-AUC, Log Loss, Brier, ECE**.
- Eleccion del **modelo ganador** que pasa a la siguiente fase, justificada por la metrica elegida.

1.2 Calibracion (variacion al notebook 10)

- **Diagnostico previo:** para el modelo ganador, dibujar el **reliability diagram** y calcular **Brier, ECE** y **Log Loss** sobre test. Comentar si el modelo esta sobre-confiado, infra-confiado o ya razonablemente calibrado.
- **Decision:** tomar de forma **explicita** la decision de calibrar o no, **justificada** con los numeros del diagnostico.

- **Si se decide calibrar:** aplicar un **metodo fiable** (**sigmoid** / **isotonic** / Venn-Abers) y verificar que la calibracion mejora (**ECE**, **Brier**) **sin degradar** la discriminacion (**AUC**, **MCC**).
- **Si se decide NO calibrar:** explicar por que el modelo no necesita calibracion, argumentado con datos.

1.3 Medida de incertidumbre y derivacion a un agente

Hasta aqui tenemos una **probabilidad puntual** p para cada cliente. Pero esa probabilidad puede ser, ella misma, **incierta**: dos clientes pueden tener $p = 0.55$ y, sin embargo, en uno el modelo esta razonablemente seguro de que la probabilidad real esta cerca de 0.55 y en el otro no tiene ni idea (podria estar en cualquier sitio entre 0.4 y 0.7). Queremos detectar este segundo caso y **derivar a un agente humano**.

- **Pregunta abierta a responder en el notebook:**

"Tengo una probabilidad puntual de mi modelo. ¿Que necesito para medir la incertidumbre de esa probabilidad y poder derivar a un agente cuando esa incertidumbre sea alta? ¿Me sirve la calibracion clasica (sigmoid/isotonic)? ¿Que estoy obteniendo realmente con cada metodo?"

El notebook debe argumentar la respuesta. Pista: la calibracion clasica devuelve una sola probabilidad calibrada, **sin intervalo**, asi que no informa de cuanto de incierta es. Hace falta un metodo que devuelva un **intervalo de probabilidad** $[p_{low}, p_{high}]$. El alumno debe identificar y aplicar uno (la opcion natural en este contexto, y la que esperamos, es **Venn-Abers Predictors**, que ademas son automaticamente calibrados; tambien se acepta cualquier otra justificada — bootstrap del modelo, ensemble disagreement, etc.).

- **Implementacion:**
 - Aplicar el metodo elegido al modelo final (por ejemplo **venn_avers.VennAbersCalibrator** o equivalente). Para cada sample de test obtener p_{low} y p_{high} .
 - Reportar la **distribucion de la anchura** $p_{high} - p_{low}$ sobre test (histograma, percentiles).
- **Politica de derivacion:**

Si $|p_{high} - p_{low}| > 0.2 \rightarrow decision = "agent"$. En caso contrario $\rightarrow decision = "auto"$.

Es decir, solo dejamos que el modelo decida cuando el intervalo de probabilidad es **estrecho** (modelo seguro de su propia probabilidad).

- **Reflexion adicional:** bajo esta politica, ¿hace falta calibrar puntualmente con sigmoid/isotonic, o el propio metodo de intervalo (Venn-Abers) ya garantiza calibracion sobre los casos que se quedan en **auto**? Razonar.
- **Metricas con derivacion** sobre test:
 - **% de casos derivados al agente.**
 - Accuracy / Precision / Recall **solo sobre los casos auto-decididos.**
 - Comparar con las metricas globales del paso 1.1 para mostrar el trade-off cobertura/precision.

1.4 Persistencia del modelo

Guardar como ultimo paso del notebook:

- `models/practica2_model.pkl`: pipeline final (modelo + calibrador + objeto de intervalo Venn-Abers, o lo que se haya elegido).
- `models/feature_schema.json`: nombres y tipos de features que espera el modelo.
- `models/top5_features.json`: las 5 features mas relevantes segun un Random Forest entrenado sobre el dataset filtrado (las que pedira la web).
- `models/feature_defaults.json`: valor por defecto (mediana / moda en train) para el resto de features.

Estos cuatro ficheros son los que el Repo 2 (API) y el Repo 3 (Web) consumiran.

Repo 2 - API local con FastAPI

Repositorio independiente con una API REST en **FastAPI**, ejecutable en local con `uv` y `uvicorn`. **No depende del codigo del Repo 1**: solo consume el `.pkl` y los `.json` que produce el notebook.

Dos endpoints:

1. **POST /model/upload**: sube un nuevo artefacto del modelo al servicio.
 - Acepta el `.pkl` como `multipart/form-data` (o una ruta local).
 - Lo carga, valida que tiene los metodos esperados y lo deja activo en memoria.
 - Devuelve la version y un timestamp.
2. **POST /predict**: devuelve la **probabilidad de impago con intervalo de incertidumbre** y la decision.
 - Recibe un JSON con las features del cliente.
 - Devuelve:

```
{
  "p_default": 0.31,
  "p_low": 0.22,
  "p_high": 0.41,
  "decision": "auto" | "agent",
  "reason": "p_high - p_low > 0.2"
}
```

- Regla: `decision = "agent"` si `p_high - p_low > 0.2`. En caso contrario, `decision = "auto"`.

Requisitos tecnicos:

- Stack fijo: **FastAPI + uvicorn**, gestionado con `uv`. Se arranca en local con:

```
uv run uvicorn api.main:app --reload --port 8080
```

- **README** con ese comando + ejemplos de invocacion con `curl` para los dos endpoints + ejemplo de payload.
- Habilitar **CORS** para que la web del Repo 3 pueda llamar al endpoint desde el navegador.
- Aprovechar la doc automatica de FastAPI en `http://localhost:8080/docs` (Swagger UI) como evidencia de que la API funciona.
- **Dockerfile opcional** (suma puntos pero no es obligatorio).

Repo 3 - Web (proyecto nuevo, generada con ayuda de IA)

Repositorio independiente con una **mini web sencilla, sin frameworks sofisticados** (nada de React, Vue, Angular o similares). Lo natural y lo que esperamos:

- **HTML + CSS + JavaScript planos** (un `index.html`, un `styles.css` opcional, un `app.js`), abrible directamente con `open index.html` o servido con `python -m http.server`.
- O equivalentemente: una sola pagina HTML servida desde un mini-servidor estatico.

Se valora positivamente que la web haya sido **generada con ayuda de un asistente de IA** (Claude, ChatGPT, Cursor, etc.). En el **README** el alumno debe:

- Indicar **que herramienta de IA** se uso.
- Pegar (al menos) el **prompt principal** con el que se genero la web, para evidenciar el uso adecuado de IA en la fase de prototipado de UI.

Funcionalidad:

- Formulario con **solo 5 campos**: las **5 features mas relevantes** segun la importancia de un `RandomForestClassifier` (calculadas en el Repo 1 y persistidas en `top5_features.json`). El resto de features se rellenan en cliente con `feature_defaults.json` antes de enviar a la API.
- Boton "**Predecir**" que hace `fetch` a `POST {API_URL}/predict`.
- Muestra:
 - "**Va a pagar**" / "**No va a pagar**" segun la prediccion (umbral `p_default >= 0.5`).
 - El **riesgo** (probabilidad de impago) en porcentaje y con una **barra visual** (`<progress>` o div con anchura proporcional).
 - Si `decision = "agent"`, en lugar del resultado automatico, mostrar un aviso del tipo "**Caso incierto, derivado a un analista humano**", indicando la anchura del intervalo `[p_low, p_high]`.

Requisitos tecnicos:

- HTML/CSS/JS sin bundlers ni frameworks SPA.
- La URL de la API se configura desde un fichero `config.js` (o equivalente) con `const API_URL = "http://localhost:8080";`. No hardcodear dentro del codigo de la logica.
- **README** con:
 - Como abrir la web en local (`open index.html` o `python -m http.server 5500`).
 - La herramienta y prompt de IA usados.
 - Capturas de pantalla del resultado en los dos casos (`auto` y `agent`).

Notas tecnicas

- Repo 1 y Repo 2 usan **uv** como gestor de entorno y dependencias (igual que el repo de clase). Las dependencias nuevas (**optuna**, **venn-abers**, **fastapi**, **uvicorn**, **python-multipart** para el upload, etc.) se anaden con **uv add <paquete>** y quedan reflejadas en **pyproject.toml** / **uv.lock**.
- **Log Loss** esta disponible como **sklearn.metrics.log_loss**. **MCC** como **sklearn.metrics.matthews_corrcoef**. **ECE** se calcula a partir del reliability diagram (binning).
- Si se usa **optimizacion multi-objetivo** con Optuna:

```
study = optuna.create_study(directions=['minimize', 'maximize'])
# objective devuelve (brier_val, auc_val)
```

La eleccion del modelo final se hace sobre **study.best_trials** (frente de Pareto) con un criterio explicito.

- Para los **intervalos de probabilidad** (Repo 1, paso 1.3) la opcion natural es **Venn-Abers Predictors** (libreria **venn-abers** o equivalente), que ademas son automaticamente calibrados. Se acepta cualquier metodo alternativo justificado: bootstrap del modelo (intervalos por resampling), ensemble disagreement, MC Dropout en redes, etc.
- En la API, **no exponer datos sensibles** (no loguear el payload completo en produccion). En local pueden loguearse para debug.
- **CORS** en la API (Repo 2): habilitar **fastapi.middleware.cors.CORSMiddleware** con **allow_origins=["*"]** en local para permitir las llamadas desde la web del Repo 3.

Rubrica de puntuacion (sobre 10)

Repo 1 - Optimizacion con Optuna (2 puntos)

Criterio	Puntos	Descripcion
Metrica que combina calibracion y discriminacion	0.5	Se optimiza Log Loss, multi-objetivo o metrica combinada justificada (no solo Brier ni solo AUC).
Variacion respecto al notebook 11	0.5	Al menos un cambio justificado en sampler, pruner o espacio de busqueda.
Comparacion balanceado vs no balanceado	0.5	Cada modelo se entrena en las dos modalidades y se comparan resultados.
Tabla final y eleccion del ganador	0.5	Tabla con metricas (Accuracy, Precision, Recall, F1, MCC, AUC, PR-AUC, Log Loss, Brier, ECE) y eleccion justificada del modelo.

Repo 1 - Calibracion (1 punto)

Criterio	Puntos	Descripcion
Diagnostico (reliability diagram + ECE + Brier + Log Loss)	0.25	Se diagnostica la calibracion del modelo ganador con metricas y grafico.

Criterio	Puntos	Descripcion
Decision argumentada (calibrar o no)	0.25	Se decide calibrar o no con justificacion numerica, no por defecto.
Aplicacion correcta (si calibra) o argumentacion (si no)	0.5	Si calibra, aplica un metodo fiable y verifica que no degrada AUC/MCC. Si no, justifica solidamente.

Repo 1 - Incertidumbre y derivacion a un agente (2 puntos)

Criterio	Puntos	Descripcion
Identificacion del metodo necesario	0.5	Se responde la pregunta abierta: la calibracion clasica no basta, hace falta un metodo que devuelva intervalo (Venn-Abers o equivalente justificado).
Implementacion del intervalo <code>[p_low, p_high]</code>	0.5	Se aplica el metodo elegido y se reporta la distribucion de la anchura sobre test.
Politica de derivacion (anchura > 0.2)	0.5	Regla implementada; se reporta % derivado + metricas restringidas a casos auto vs metricas globales.
Persistencia del artefacto	0.5	El notebook guarda <code>practica2_model.pkl</code> , <code>feature_schema.json</code> , <code>top5_features.json</code> y <code>feature_defaults.json</code> .

Repo 2 - API con FastAPI (2 puntos)

Criterio	Puntos	Descripcion
Endpoint <code>/model/upload</code>	0.5	Sube y carga un nuevo <code>.pkl</code> , lo valida y lo deja activo.
Endpoint <code>/predict</code> con intervalo	1.0	Devuelve <code>p_default</code> , <code>p_low</code> , <code>p_high</code> y <code>decision</code> segun la regla <code>p_high - p_low > 0.2</code> .
Ejecucion local + CORS + Swagger	0.5	<code>uv run uvicorn ...</code> arranca, CORS habilitado, <code>/docs</code> accesible, <code>README</code> con <code>curl</code> de ejemplo.

Repo 3 - Web sencilla generada con IA (2 puntos)

Criterio	Puntos	Descripcion
Stack sencillo (HTML/CSS/JS planos, sin React/Vue/Angular)	0.5	Web abrible directamente, sin bundlers.
Uso documentado de IA para generar la web	0.5	<code>README</code> indica herramienta y prompt principal.
Top-5 features + defaults + llamada a API	0.5	Pide solo 5 inputs, completa el payload con los defaults y llama a <code>POST /predict</code> .

Criterio	Puntos	Descripcion
UI con riesgo, barra y aviso de derivacion	0.5	Muestra prediccion + probabilidad + barra; en caso agent muestra aviso en lugar del resultado automatico.

Calidad y entrega (1 punto)

Criterio	Puntos	Descripcion
Tres URLs entregadas y accesibles	0.25	Repo 1, Repo 2 y Repo 3 entregados como repositorios independientes.
README en cada repo	0.5	Cada repo tiene README propio con instrucciones de ejecucion en local y, en Repo 3, capturas + prompt de IA.
Reproducibilidad	0.25	random_state fijado, dependencias congeladas (uv.lock), instrucciones claras para reproducir Repo 1 + Repo 2 + Repo 3.

Fecha de entrega

Por determinar.

Recursos utiles

- Notebooks de referencia: [09_conformal_prediction.ipynb](#), [10_calibracion.ipynb](#), [11_optuna_tpe.ipynb](#).
- Documentacion de [Optuna](#) (samplers TPE, pruners Median/Hyperband, multi-objective).
- [sklearn.calibration.CalibratedClassifierCV](#), [sklearn.calibration.calibration_curve](#).
- [sklearn.metrics.log_loss](#), [sklearn.metrics.brier_score_loss](#), [sklearn.metrics.matthews_corrcoef](#).
- Libreria [MAPIE](#) para conformal prediction (opcional).
- Streamlit para la web: [streamlit.io](#).